UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

Student Name: Ashi Gupta **UID: 25MCA20160**
Branch: M.C.A GENERAL **Section/Group: MCA-1 A**
Semester: Second Sem **Date of Performance:2/03/2026**
Subject Name: TECHINCAL SKILLS **Subject Code:**

WORKSHEET 5

AIM: To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic. **(Company Tags: Infosys, Wipro, TCS, Capgemini)**

S/W Requirement: Oracle Database Express Edition and pgAdmin

OBJECTIVES:

- **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
- **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
- **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
- **Exception Handling:** To handle cursor-related errors and performance considerations during large-scale data iteration.

Practical / Experiment Steps

Step 0: Setup (Creation of Table)

```
CREATE TABLE Employee (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    salary NUMERIC(10,2),
    experience INT,          -- Years of experience
    performance_score INT    -- Rating out of 10
);
```

Inserting Sample Values

```
INSERT INTO Employees (emp_name, salary, experience, performance_score) VALUES
('Amit', 40000, 2, 6),
('Neha', 60000, 5, 8),
```

('Rohit', 55000, 4, 7),

('Priya', 75000, 8, 9),

('Karan', 30000, 1, 5);

Output:

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)	experience integer	performance_score integer
1	1	Amit	40000.00	2	6
2	2	Neha	60000.00	5	8
3	3	Rohit	55000.00	4	7
4	4	Priya	75000.00	8	9
5	5	Karan	30000.00	1	5

Step 1: Implementing a Simple Forward-Only Cursor

- Creating a cursor to loop through an Employee table and print individual records.

```
DO $$
DECLARE
   emp_record RECORD;      -- Variable to hold one row
   emp_cursor CURSOR FOR   -- Declare cursor
      SELECT emp_id, emp_name, salary
      FROM Employee;
BEGIN

   OPEN emp_cursor;   -- Open cursor

   LOOP
      FETCH emp_cursor INTO emp_record;   -- Fetch one row

      EXIT WHEN NOT FOUND;   -- Exit when no more rows
```

```
    RAISE NOTICE 'ID: %, Name: %, Salary: %',
                 emp_record.emp_id,
                 emp_record.emp_name,
                 emp_record.salary;
  END LOOP;
  CLOSE emp_cursor;   -- Close cursor
END $$;
```

Output:

```
NOTICE:  ID: 1, Name: Amit, Salary: 40000.00
NOTICE:  ID: 2, Name: Neha, Salary: 60000.00
NOTICE:  ID: 3, Name: Rohit, Salary: 55000.00
NOTICE:  ID: 4, Name: Priya, Salary: 75000.00
NOTICE:  ID: 5, Name: Karan, Salary: 30000.00
DO

Query returned successfully in 96 msec.
```

Step 2: Complex Row-by-Row Manipulation

- Using a cursor to update salaries based on a dynamic "Experience-to-Performance" ratio logic.

```
DO $$
DECLARE
  emp_record RECORD;

  -- Declare cursor
  emp_cursor CURSOR FOR
    SELECT emp_id, salary, experience, performance_score
    FROM Employee;

  increment_percent NUMERIC;
  new_salary NUMERIC;

BEGIN
```

```
OPEN emp_cursor;

LOOP
   FETCH emp_cursor INTO emp_record;

   EXIT WHEN NOT FOUND;

   /* Calculate increment percentage */
   increment_percent :=
      (emp_record.experience * emp_record.performance_score) / 10.0;

   /* Calculate new salary */
   new_salary :=
      emp_record.salary +
      (emp_record.salary * increment_percent / 100);

   /* Update salary */
   UPDATE Employee
   SET salary = new_salary
   WHERE emp_id = emp_record.emp_id;

   /* Display update message */
   RAISE NOTICE
   'Employee ID: %, Increment: %%%, New Salary: %',
   emp_record.emp_id,
   increment_percent,
   new_salary;

END LOOP;
CLOSE emp_cursor;
```

END $$;

Output:

```
NOTICE:  Employee ID: 1, Increment: %1.2000000000000000, New Salary: 40480.000000000000000000
NOTICE:  Employee ID: 2, Increment: %4.0000000000000000, New Salary: 62400.000000000000000000
NOTICE:  Employee ID: 3, Increment: %2.8000000000000000, New Salary: 56540.000000000000000000
NOTICE:  Employee ID: 4, Increment: %7.2000000000000000, New Salary: 80400.000000000000000000
NOTICE:  Employee ID: 5, Increment: %0.50000000000000000000, New Salary: 30150.0000000000000000000000
DO

Query returned successfully in 54 msec.
```

Step 3: Exception and Status Handling

- Ensuring the cursor handles empty result sets or termination signals gracefully.

```
DO $$
DECLARE
    emp_record RECORD;

    emp_cursor CURSOR FOR
        SELECT emp_id, salary
        FROM Employee;

BEGIN

    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO emp_record;

        EXIT WHEN NOT FOUND;

        -- Intentionally raise exception if salary < 35000
```

```
    IF emp_record.salary < 35000 THEN
        RAISE EXCEPTION
        'Salary too low for Employee ID: %',
        emp_record.emp_id;
    END IF;

    RAISE NOTICE
    'Processed Employee ID: %, Salary: %',
    emp_record.emp_id,
    emp_record.salary;

  END LOOP;

  CLOSE emp_cursor;

EXCEPTION
  WHEN OTHERS THEN
    RAISE NOTICE 'Exception Caught Successfully!';
    RAISE NOTICE 'Error Message: %', SQLERRM;
    RAISE NOTICE 'Execution stopped safely.';

END $$;
```

Output:

```
NOTICE:  Processed Employee ID: 1, Salary: 40965.76
NOTICE:  Processed Employee ID: 2, Salary: 64896.00
NOTICE:  Processed Employee ID: 3, Salary: 58123.12
NOTICE:  Processed Employee ID: 4, Salary: 86188.80
NOTICE:  Exception Caught Successfully!
NOTICE:  Error Message: Salary too low for Employee ID: 5
NOTICE:  Execution stopped safely.
DO

Query returned successfully in 52 msec.
```

END $$;

Outcomes:

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.